CHAPTER 02

Scale From Zero To Millions Of Users

Evolve a single-server app into a globally distributed system, one bottleneck at a time.

1. Problem statement

Design a system that begins with one user on one machine and grows to serve millions of users. There is no fixed product: the interviewer wants to see that you can **find the next bottleneck, apply the right technique, and explain the trade-off**. This chapter is the toolbox every later design problem draws from.

Why interviewers ask it: It tests whether you know the standard building blocks (load balancer, replication, cache, CDN, stateless tier, queues, sharding) and, more importantly, **when** each one is justified.

2. Requirements

Functional

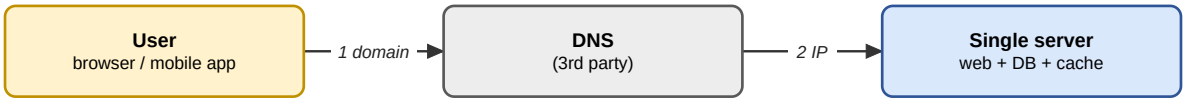
- Serve web and mobile clients through a domain name (web: `www.site.com`, API: `api.site.com`).
- Store and retrieve user data durably.
- Keep working when a server, database or whole data center fails.

Non-functional

- **Scalability:** add capacity horizontally without a rewrite.
- **Availability:** no single point of failure (SPOF) at any tier.
- **Low latency:** users anywhere get fast responses (cache, CDN, geo-routing).
- **Operability:** logs, metrics and automation so a small team can run it.

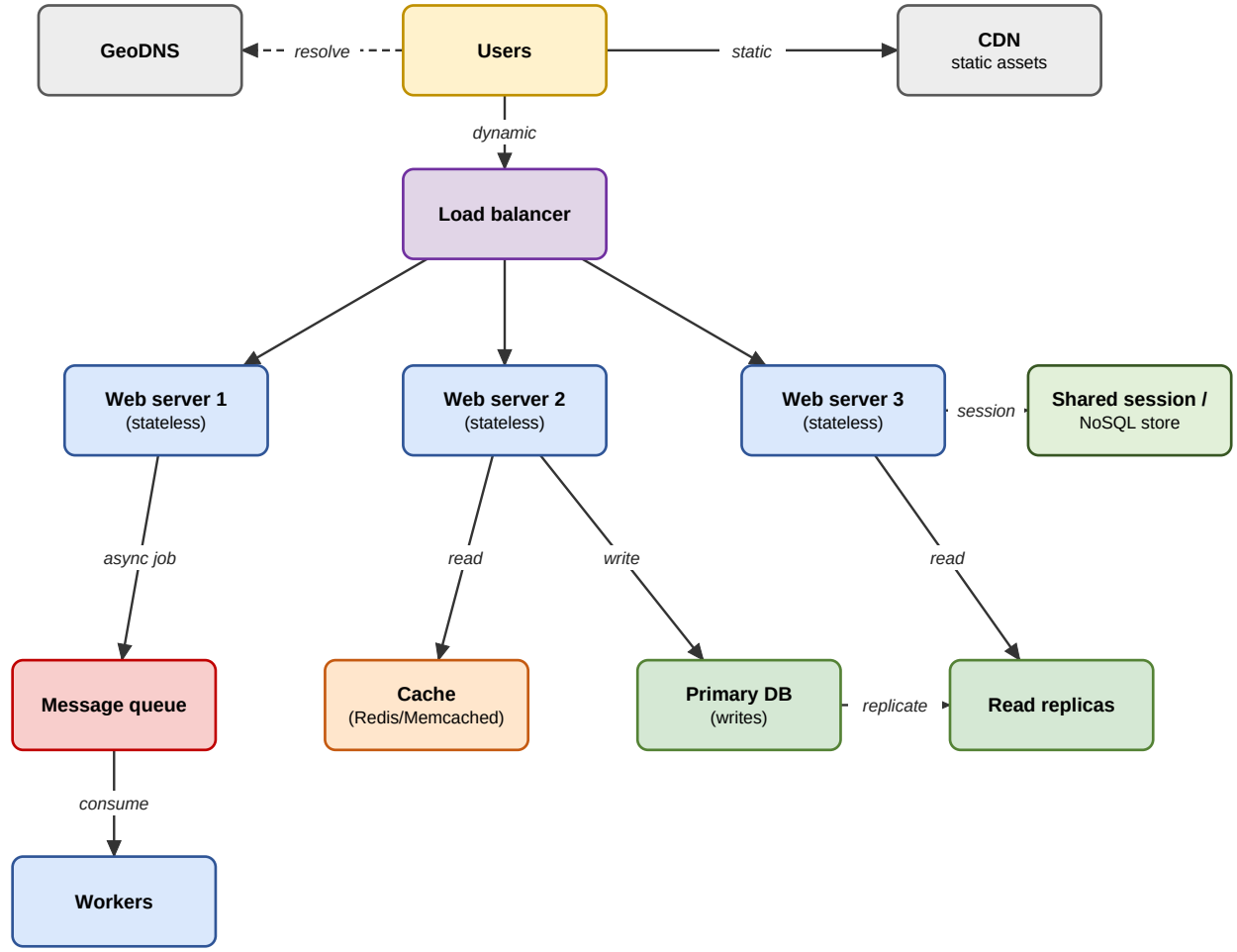
3. High-level design

Stage 1 - single server (everything on one box)



DNS resolves the domain to an IP, then the client sends HTTP requests straight to the server.

Stage N - the full multi-data-center architecture



Every tier is replicated; the web tier is stateless; slow work goes through a queue.

4. Deep dive

Step 1 -> 2: separate the database from the web tier

- Independent scaling and failure isolation: web servers and DB have different resource profiles.
- **Relational (MySQL/Postgres)** is the default: joins, transactions, mature tooling.
- **NoSQL** (key-value, document, column, graph) when you need super-low latency, unstructured data, huge volume, or only need to serialise/deserialise data (e.g. sessions, logs, activity feeds).

Vertical vs horizontal scaling

- **Vertical (scale up):** bigger machine. Simple, but hard cap, expensive, still a SPOF.
- **Horizontal (scale out):** more machines. Needed for large-scale apps; requires a load balancer and a stateless tier.

Load balancer

- Spreads traffic across web servers using **private IPs**, so servers are not reachable directly from the internet.
- Removes the web-tier SPOF: if server 1 dies, traffic shifts to server 2; add servers to absorb spikes.
- Algorithms: round robin, least connections, weighted, IP-hash (sticky). Health checks remove bad nodes.
- The LB itself must be redundant (active-passive or managed multi-AZ LB).

Database replication

- **Primary** handles writes; **replicas** handle reads (most apps are read-heavy, e.g. 10:1).
- Benefits: read throughput, reliability (backups in other locations), availability.
- Replica down: reads go to other replicas or the primary. **Primary down:** promote a replica (data may lag - recover with scripts; multi-primary / circular replication is more complex).
- Replication lag means a read right after a write can be stale -> read-your-writes may need primary reads.

Cache tier

- Cache = fast in-memory store for results of expensive queries or frequently read data. Pattern: **read-through / cache-aside** - check cache, on miss read DB, then populate cache.
- Considerations: use for read-heavy, rarely changing data; **expiration (TTL)** policy; **consistency** between DB and cache; **mitigating failures** (multiple cache nodes, overprovision memory); **eviction** (LRU, LFU, FIFO).
- Hot-key and cache-stampede protection: request coalescing, jittered TTLs.

CDN

- Geographically distributed servers that serve **static content** (images, JS, CSS, video).
- Flow: user asks CDN -> miss -> CDN fetches from origin, caches with a TTL (`Cache-Control: max-age`) -> hit afterwards.
- Concerns: cost (do not cache rarely used content), expiry TTL, **CDN fallback** to origin, invalidation (API or object versioning like `image.png?v=2`).

Stateless web tier

- Stateful servers remember client data (session) between requests -> sticky sessions, hard to autoscale or fail over.
- Stateless: move state (sessions) to a **shared persistent store** (Redis / NoSQL). Any server can serve any request.
- Enables **autoscaling**: add/remove servers on CPU or request-rate metrics.

Multiple data centers

- **GeoDNS** routes users to the nearest data center; if one DC is down, all traffic fails over to a healthy DC.
- Challenges: **traffic redirection**, **data synchronisation** (replicate across DCs; failover may hit a DC with no local data), **test and deploy** consistently across locations (automated tooling).

Message queue

- Durable buffer that **decouples** producers and consumers (e.g. photo upload -> thumbnail workers).
- Producers and consumers scale independently; the queue absorbs bursts; consumers can lag without losing messages.

Logging, metrics, automation

- **Logging:** per-server error logs are fine for small systems; aggregate centrally at scale.
- **Metrics:** host-level (CPU, memory, disk I/O), aggregated-level (DB tier performance), key business metrics (DAU, revenue, retention).
- **Automation:** CI/CD, build, test, deploy pipelines improve productivity.

Database scaling beyond replication: sharding

- Vertical DB scaling has hardware limits and SPOF risk -> **horizontal scaling = sharding**: split data across DBs, each a shard with the same schema.
- Pick a **sharding key** with even distribution (e.g. `user_id % N`). Key choice is the most important decision.
- Challenges: **resharding** when a shard fills or distribution is skewed (consistent hashing helps - chapter 06), **celebrity / hotspot** problem (dedicated shard), **joins and denormalisation** (cross-shard joins are hard).

Millions of users and beyond - checklist

- Keep the web tier stateless. Build redundancy at every tier. Cache data as much as you can.
- Support multiple data centers. Host static assets in a CDN. Scale the data tier by sharding.
- Split tiers into individual services (microservices). Monitor and automate.

5. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
SQL vs NoSQL	Relational: ACID, joins, flexible queries / NoSQL: horizontal scale, low latency, flexible schema	Default to SQL; choose NoSQL for huge scale, simple access patterns, or unstructured data.
Cache vs read replica	Cache: in-memory, sub-ms, extra consistency problem / Replica: simple, still disk-bound	Replicas first for read scale; add cache for hot data and tight latency.
Sticky sessions vs stateless	Sticky: easy start / Stateless: shared session store	Stateless is the right answer for autoscaling and resilient failover.
Vertical vs horizontal	Bigger box / more boxes	Vertical early (simplicity); horizontal when you hit limits or need HA.

6. Likely follow-up questions

- What happens if the load balancer fails? (redundant LB pair, managed LB across AZs)
- How do you keep cache and DB consistent on writes? (invalidate on write, short TTL, write-through)
- How would you pick a sharding key and reshard with zero downtime?
- How do you keep multi-DC data in sync, and what is the failover story for in-flight writes?
- When would you introduce microservices, and what do you lose when you do?

7. Common mistakes

- Jumping straight to the final architecture without showing the evolution and the bottleneck that justifies each step.
- Forgetting the SPOF of the load balancer, cache, or queue itself.
- Treating the cache as the source of truth.
- Sharding before exhausting replication, caching, and query tuning.

8. 60-second recap / practice checklist

- Single server -> split DB -> LB + many stateless web servers -> replicate DB -> cache + CDN -> shared session store -> multi-DC (GeoDNS) -> queue + workers -> logging/metrics/automation -> shard the DB.
- For every component, be ready to state: **what problem it solves, what new problem it creates**.
- Practice: draw the final diagram from memory in under two minutes and narrate each arrow.